

LoRaMesher: An open-source library for multi-hop LoRa mesh networks

Joan Miquel Solé, Felix Freitag, Roger Baig* 

Universitat Politècnica de Catalunya–BarcelonaTech (UPC), Barcelona, Spain

ARTICLE INFO

Keywords:

LoRa
Mesh networks
Routing
LPWAN
IoT

ABSTRACT

LoRaMesher is an open-source C++ library that enables multi-hop mesh networks over LoRa communication on ESP32 microcontroller boards. The library implements a proactive distance-vector routing protocol that maintains routing tables on each node and provides unicast, broadcast, and reliable message delivery, including fragmentation and reassembly for large payloads. LoRaMesher leverages RadioLib for radio access and FreeRTOS for task scheduling, and integrates with PlatformIO for development. By allowing LoRa devices to communicate directly in a mesh network, rather than via LoRaWAN gateways, the library enables fully distributed applications on low-power IoT nodes. This paper describes the LoRaMesher software, its architecture, main functionalities, and usage examples, and discusses its relevance for research and applied IoT deployments.

Code metadata

Nr.	Code metadata description	Metadata
C1	Current code version	v0.0.11-alpha
C2	Permanent link to code/repository	https://github.com/LoRaMesher/LoRaMesher
C3	Permanent link to Reproducible Capsule	Not available
C4	Legal Code License	MIT License
C5	Code versioning system	Git (GitHub)
C6	Languages, tools, services	C++; Arduino/ESP-IDF; RadioLib; FreeRTOS; PlatformIO; Doxygen
C7	Compilation requirements	ESP32 boards with LoRa radios (SX127x/SX126x/SX1280); PlatformIO or Arduino IDE
C8	Link to documentation	https://github.com/LoraMesher/LoraMesher
C9	Support email or contact	GitHub Issues page

1. Motivation and significance

LoRa mesh networks have recently been proposed for Internet of Things (IoT) scenarios beyond LoRaWAN [1]. In such networks, any two nodes can send messages to each other, and each node may act as a source or a destination. By leveraging multi-hop capacity, LoRa mesh networks can extend connectivity among LoRa nodes across large geographic areas [2].

Existing IoT applications that rely on LoRa communication typically use the traditional LoRaWAN architecture, which defines a star topology. In this topology, LoRa end-nodes communicate only with gateways in a single hop, and not with other end-nodes [3]. LoRaWAN is usually

well suited to support sensing applications, where monitoring data is periodically communicated from the sensor node on the uplink to the gateway. However, LoRaWAN does not support LoRa mesh communication with node-to-node and multi-hop communication capacity. Hence, new types of distributed applications that run directly on low-power IoT nodes cannot be supported.

LoRa networking scenarios that are different from LoRaWAN have been investigated in several works, for example, in [4–6], but most of these works target fundamental issues with results obtained from simulations rather than from software that is deployed on real nodes. Deployments of LoRa mesh networks are presented in [7,8], contributing

* Corresponding author.

Email address: roger.baig@upc.edu (R. Baig).

<https://doi.org/10.1016/j.softx.2026.102570>

Received 21 December 2025; Received in revised form 2 February 2026; Accepted 13 February 2026

Available online 20 February 2026

2352-7110/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

to performance understanding for the specific application. While the related works show that one of the application targets of LoRa mesh networks is traditional monitoring applications, where the multi-hop capacity is added to forward data to a gateway, the node-to-node communication capability of LoRa mesh networks can also be highlighted. This capacity can become relevant, for instance, in the increasingly popular area of Edge AI, which includes Machine Learning (ML) on microcontrollers in resource-constrained IoT environments [9]. Remote intelligent nodes containing neural networks host complex applications that become destinations to which data has to be sent, e.g., for over-the-air model updates [10] or federated on-device model training [11]. LoRa mesh networks, with multi-hop and node-to-node communication, could provide the networking substrate on which such applications could be implemented, deployed, and, through operational implementations, be evaluated in real environments.

One of the most widespread open-source, community-driven solutions today is Meshtastic [12]. Indeed, Meshtastic provides a text-messaging application built on top of its LoRa mesh network. Each participant uses a smartphone for the user interface and a LoRa node connected to the mesh, with communication between the two handled over Bluetooth. However, the Meshtastic mesh is tailored for its specific messaging application. Unlike LoRaMesher, the Meshtastic networking substrate is not offered as a standalone library, which poses challenges for developers to leverage the implementation for a customized purpose. Protocol-wise, Meshtastic and LoRaMesher are different since Meshtastic uses packet flooding rather than routing as in LoRaMesher.

There is also a limited number of commercial LoRa mesh network solutions. One provider is Neocortec, which offers the NeoMesh solution [13]. NeoMesh offers a time-synchronized mesh protocol designed for battery-powered devices. NeoMesh is closed source and tightly coupled to its own hardware, which limits flexibility to be used for research and development by others. Another commercial solution is ORAMA-Net from Orama-systems [14]. While ORAMA-Net supports multi-hop LoRa networking, its application model resembles LoRaWAN, in which the destination is the gateway rather than mesh nodes. Like NeoMesh, ORAMA-Net is closed source.

LoRaMesher¹ addresses the lack of open-source software in the community for operational LoRa mesh network implementations. It was developed through academic research at the university. Its theoretical foundations were designed in a 2021 PhD thesis [15]. Given its academic origins, LoRaMesher has been an open-source project from the outset. It is presented as a library that can easily be integrated into third-party applications and that can be deployed on ESP32-based microcontroller boards with LoRa radio. The configuration can be customized for the specific deployment and application. Relevant scientific publications about LoRaMesher's protocol, design, performance, and evaluation are openly available [16–19], and an active Discord community provides practical support.

With LoRaMesher, the community obtains an operational LoRa mesh network implementation that includes automatic neighbor discovery, message queues for incoming and outgoing messages, dynamic routing table updates, multi-hop routing, fragmentation of large messages, and reliable message delivery.

2. Software description

2.1. Software architecture

LoRaMesher is a C++ library that enables packet exchange between nodes in a LoRa mesh network using a proactive distance-vector routing protocol. The library uses FreeRTOS to create several tasks responsible for packet reception, transmission, processing, routing, and application-level data handling. RadioLib is used to interface with the LoRa radio hardware. The library runs on ESP32 microcontroller boards.

The LoRaMesher implementation is organized around the LoRaMesher class in `LoRaMesher.cpp`. As a library, it is included in an application through `LoRaMesher.h` and used in `main.cpp`, as shown in Fig. 1. The `main.cpp` code obtains a single static object of the LoRaMesher class. The class implements six FreeRTOS tasks. The scheduling of each task is detailed in [16]. Within `LoRaMesher.cpp`, three queues are created to support basic packet processing: a queue for received packets, a queue for packets to be sent, and a queue that delivers packets to the application defined in `main.cpp`. The queues provide message buffers for the tasks.

Basic packet processing in LoRaMesher handles low-level packet management at each node. For this, an application sends a packet by calling `createPacketAndSend()` from `main.cpp`, indicating the destination and the payload. When the `SentTask` is scheduled, it retrieves the packet from the send-queue and transmits it. Incoming LoRa packets at a node are first placed in the received-packets queue, managed by the `ReceiveTask`. The `ProcessTask` then inspects the packet to determine its destination. If the packet is addressed to the local application, it is placed in the application-delivery queue; otherwise, it is placed in the send-queue for transmission to the next hop.

In the reliable transfer mode, invoked from the application in `main.cpp` via the `sendReliable()` function, additional queues are instantiated depending on the node's role. These are queues for transmitting or receiving large messages. The `QueueManagerTask` manages these queues and timeouts in the event of packet loss. Reliable transfer is implemented on top of the basic packet processing on the source and destination nodes. Reliable transfer mode operates end-to-end. Therefore, it is applied to the message at both the source and destination nodes. Intermediate nodes in multi-hop paths that forward packets but do not deliver them to the application perform basic packet processing.

The implementation of the FreeRTOS tasks is supported by additional data structures, hardware-specific details, and low-level packet-processing functions, which are organized into folders within the LoRaMesher repository. The `entities` folder defines low-level packet and routing table structures. The `modules` folder contains hardware-specific implementations for the supported LoRa radio. The utility functions used by the tasks reside in the `service` folder. Finally, the `utilities` folder contains a linked-list implementation that serves as the basis for the various internal queues.

The main configuration file of LoRaMesher is `BuildOptions.h` (Code 1). It allows developers to set LoRa parameters such as Spreading

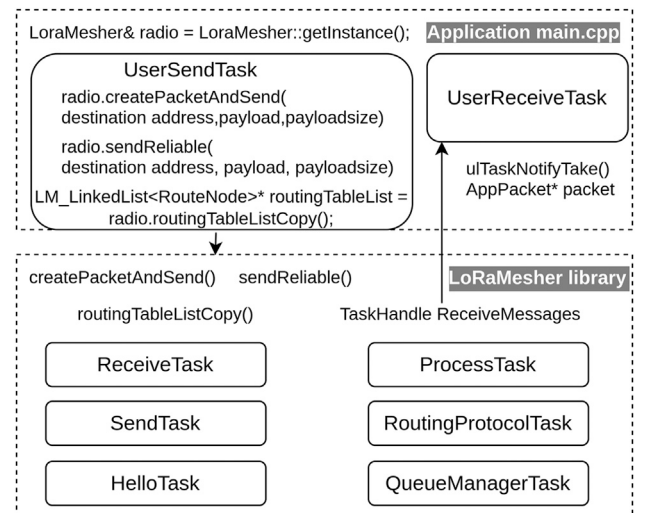


Fig. 1. LoRaMesher library and application integration.

¹ <https://github.com/LoRaMesher/LoRaMesher>

```

// LoRa band definition
#define LM_BAND 869.900F
#define LM_BANDWIDTH 125.0
#define LM_LORASF 7U
#define LM_CODING_RATE 7U
#define LM_PREAMBLE_LENGTH 8U
#define LM_POWER 6
#define LM_DUTY_CYCLE 100
...
//MAX packet size per packet in bytes.
#define LM_MAX_PACKET_SIZE 100
...
// Routing packet periodicity in seconds
#define HELLO_PACKETS_DELAY 120
...
//Maximum times that a sequence of packets reaches the
    timeout
#define MAX_TIMEOUTS 10

```

Code 1. Code fragment from the `BuildOptions.h` configuration file.

Factor (SF), transmission power, and duty cycle. It is also where parameters for LoRaMesher's routing protocol, reliable transfer, and message size limit can be customized.

2.2. Software functionalities

LoRaMesher features distinct functions that together deliver a complete LoRa mesh networking service for distributed applications on IoT nodes.

1. Automatic neighbor discovery and routing table maintenance: Each LoRaMesher node has the *HelloTask* that periodically broadcasts the routing table in a routing message. The *RoutingProtocolTask* deletes expired node routes and any corresponding open connections.
2. Broadcast and unicast packet delivery: LoRaMesher uses unicast transmission for data packets, which may traverse multiple hops to reach their destination. Broadcast is used only for routing packets.
3. End-to-end reliability with acknowledgements: When the application calls *sendReliable()*, messages are delivered with end-to-end acknowledgments to ensure reliable transmission.
4. Fragmentation and reassembly of large messages: If an application sends a message with *sendReliable()* that exceeds the maximum packet size, LoRaMesher automatically fragments it into smaller data packets that fit into the payload size defined by `LM_MAX_PACKET_SIZE` in the `BuildOptions.h` file. At the destination, the fragments are reassembled before delivery to the application.
5. Operation in dynamic traffic scenarios through LoRaMesher node's queue management on the nodes: By using queues, packets are held at a node until the next task to process the packet is scheduled. This procedure prevents loss of LoRa packets at the nodes during temporary periods of high traffic. To support this behavior, the *ReceiveTask* is assigned the highest priority.

Given these features, LoRaMesher, as a network service, supports diverse and concurrent data flows between any nodes in a multi-hop LoRa mesh network. Dynamic routing table updates enable the network to scale up and down without manual configuration. Reliable, large-payload transfer mechanisms allow the delivery of even large messages over multiple hops to a destination.

2.3. Sample code snippet analysis

LoRaMesher can be integrated into a wide range of applications, including environmental and industrial monitoring, smart-city deployments, emergency communication networks, asset tracking, and connectivity in infrastructure-poor areas. We exemplify the use of LoRaMesher with an application that periodically sends an incremental counter value from each node in the mesh network to all other nodes.

The following code snippets are based on the *Counter* example² from the LoRaMesher repository. We examine the relevant code fragments from `main.cpp` to show how the application initializes LoRaMesher, sends and receives packets. In `main.cpp`, the two FreeRTOS tasks *UserReceiveTask* and *UserSendTask* introduced in Section 2.1, are created (Fig. 1).

Code 2 shows the beginning of `main.cpp`, where the application obtains the object of the LoRaMesher class. It is important to note that `main.cpp` has only one object of it, adhering to the Singleton pattern. Next, the application defines a custom payload containing a counter value.

```

#include "LoraMesher.h"

//Get LoRaMesher class
LoraMesher& radio = LoraMesher::getInstance();

struct dataPacket {
    uint32_t counter = 0;
};

//payload to be sent
dataPacket* helloPacket = new dataPacket;

```

Code 2. Code fragment from `main.cpp` that obtains the LoRaMesher object and defines the payload to be sent.

Code 3 shows the setup of the LoRaMesher object done by the *setUpLoraMesher()* function in `main.cpp`. It sets the hardware parameter configuration of the object. A *UserReceiveTask* is created for receiving packets addressed to the application at this node. Note that the task handler is provided to LoRaMesher. Finally, LoRaMesher is started.

² <https://github.com/LoRaMesher/LoRaMesher/tree/main/examples/Counter>

```

void setupLoraMesher() {
    //Get the configuration of the LoRaMesher
    LoraMesher::LoraMesherConfig config = LoraMesher::
        LoraMesherConfig();

    //Set the configuration of LoRaMesher (e.g., TTGO T-BEAM
        v1.1)
    config.loraCs = 18;
    config.loraRst = 23;
    config.loraIrq = 26;
    config.loraIo1 = 33;

    config.module = LoraMesher::LoraModules::SX1276_MOD;

    //Init the loramesher with a configuration
    radio.begin(config);

    //Create the receive task and add it to the LoRaMesher
    createReceiveMessages();

    //Set the task handle to the LoRaMesher
    radio.setReceiveAppDataTaskHandle(
        receiveLoRaMessage_Handle);

    //Start LoRaMesher
    radio.start();
}

```

Code 3. Code fragment from `main.cpp` that configures and starts `loramesher`, and creates the receive task.

The `processReceivedPackets()` function, shown in [Code 4](#), is executed by the `UserReceiveTask`. It runs whenever the task is notified of a new packet reception via `ulTaskNotifyTake`. The payload of the received packet is then made available for further processing by the application.

[Code 5](#) shows the `sendLoRaMessage()` function, executed by the `UserSendTask`. The task is created in `main.cpp`. In our application, the function reads the routing table to obtain the addresses of known nodes. Once entries are available, the counter value is sent periodically to all other nodes. Since the counter value fits into a LoRa packet and packet delivery acknowledgement is not required in the example, the function uses `createPacketAndSend()`.

By integrating these code snippets into a complete `main.cpp` file, a `LoRaMesher` node is created that periodically sends packets containing its current counter value to all discovered nodes, forwards packets not addressed to it, and, at the application layer, processes the packets for which the node is the destination.

3. Illustrative examples

To illustrate the system's operation, we flash `LoRaMesher's CounterAndDisplay` example³ on three T-Beam ESP32-based microcontroller boards. The example runs the `main.cpp` file that contains the code snippets of [Section 2.3](#) and adds a task to operate the onboard display. [Fig. 2](#) shows the three boards. The OLED displays demonstrate that the nodes have already sent several counter values.

Next, we flash `LoRaMesher's LargePayload` example⁴ on four ESP32 microcontroller boards. The `LoRaMesher` addresses of these nodes are 0xF020, 0xE2B0, 0x2F14, and 0xC4F0.

³ <https://github.com/LoRaMesher/LoRaMesher/tree/main/examples/CounterAndDisplay>

⁴ <https://github.com/LoRaMesher/LoRaMesher/tree/main/examples/LargePayload>

The payload to be sent in this example consists of a sequence of fixed incremental numbers repeated six times to clearly exceed the maximum packet size defined by `LM_MAX_PACKET_SIZE` 100 in `BuildOptions.h`. The application invokes the `sendReliable()` function in `main.cpp`, which enables fragmentation, reliable transfer, and reassembly at the destination node.

The detailed operation of `LoRaMesher` can be observed by examining the debug messages printed to each node's serial port. [Fig. 3](#) shows the current routing table of node 0xF020. The table confirms that the node has correctly discovered the other three nodes, with metric values indicating that all nodes are one hop away. The logs also show the temporal order of `LoRaMesher's` internal operations. First, a packet is received, and then the header is inspected. The broadcast address 0xFFFF indicates that the packet is a routing packet from node 0x2F14. The packet payload is processed, and the node's routing table is updated. After processing, the packet is deleted.

[Fig. 4](#) illustrates the capacity of `LoRaMesher` to transmit large payloads. The log messages indicate that an 840-byte payload, sent from node 0xE2B0, was successfully received by the application at node 0xF020.

[Fig. 5](#) shows the reception of a data packet with segment number 10 of a large payload at node 0x2F14. The node determines the packet's destination, adds it to the internal queue that manages this reliable data transfer, and prepares the corresponding acknowledgement to be sent to the source.

[Fig. 6](#) shows how node 0xC4F0 prepares a large data transfer destined for node 0xE2B0. When the application calls the `sendReliablePacket()` function, `LoRaMesher` fragments the 840-byte payload into data packets that fit within the configured maximum packet size. The packets are numbered and enqueued. The `SendTask` transmits the packets. The `QueueManagerTask` manages timeouts for reliable data transfer.

The examples demonstrate the features of `LoRaMesher` described in [Section 2.2](#). It can be seen how a `LoRaMesher` node dynamically obtains a routing table that enables packet forwarding across the network,

```

void processReceivedPackets(void*) {
    for (;;) {
        /* Wait for the notification of
        processReceivedPackets and enter blocking */
        ulTaskNotifyTake(pdPASS, portMAX_DELAY);
        led_Flash(1, 100); //one quick LED flashes to
        indicate a packet has arrived

        //Iterate through all the packets inside the Received
        User Packets Queue
        while (radio.getReceivedQueueSize() > 0) {
            Serial.println("ReceivedUserData_TaskHandle
            notify received");
            Serial.printf("Queue receiveUserData size: %d\n",
            radio.getReceivedQueueSize());

            //Get the first element inside the Received User
            Packets Queue
            AppPacket<dataPacket>* packet = radio.
            getNextAppPacket<dataPacket>();

            //Print the data packet
            printDataPacket(packet);

            //Delete the packet when used. It is very
            important to call this function to release the
            memory of the packet.
            radio.deletePacket(packet);
        }
    }
}

```

Code 4. Code fragment from `main.cpp` that implements the receive function.

and how payloads of varying sizes can be transmitted between nodes. For small messages, applications can use either basic transfer or reliable transfer. For large messages, data fragmentation and reliable transfer are applied. The examples also illustrate that LoRaMesher can handle multiple data flows concurrently.

4. Impact

For researchers and developers, LoRaMesher is one of the few open-source LoRa mesh network implementations that are ready to run on real devices. For the research community, it provides a practical tool for conducting experiments on real LoRa mesh networks, enabling researchers to compare or validate simulation results against real-world performance, thereby increasing the Technology Readiness Level (TRL) of proposed solutions and facilitating their transfer to industry. For application developers, LoRaMesher's TRL (we estimate it at level 6 based on our experimentation in [16,18,19]) is sufficiently high for adapting the code and customizing it toward operational scenarios, e.g., by extending `main.cpp` or applying specific configurations in `BuildOptions.h`.

Knowledge about LoRaMesher is accessible to everyone, as the major scientific publications describing the system are open access. In [16], the design and implementation of LoRaMesher are presented, including protocol details, packet formats, and performance results with Packet Delivery Ratio (PDR) values for different topologies. In [17], the design space of the LoRaMesher protocol is analyzed through simulations. The work in [18] describes the usage of LoRaMesher as a networking substrate for distributed IoT applications. Experiments run over 24 hours with 14 nodes demonstrate the operational stability of LoRaMesher.

Finally, LoRaMesher's large and reliable payload mechanism is introduced in [19]. Experiments conducted with up to 10 nodes characterize the performance and operation of the mechanism in different situations.

LoRaMesher has evolved into a community software project with active developers and users. Provided as open-source software, the default release of LoRaMesher can be customized and extended for research and practical uses. Based on our experience with LoRaMesher development and deployment, we outline in the following some directions for usability and possible extensions. The library is built for ESP32 microcontrollers and uses FreeRTOS tasks, which are popular, but alternative MCU hardware platforms and embedded RTOSes exist. In our experience, porting the implementation to other platforms would require a significant effort. In some scenarios, it could be desirable to enable a LoRaMesher node to support both LoRa mesh and LoRaWAN communication. Such an extension appears feasible, for example by adding a second LoRa module to the node and leveraging the existing LoRaMesher implementation by adding a task to handle LoRaWAN communication.

LoRaMesher nodes implement asynchronous operation and are expected to be permanently powered on to send, receive, and relay messages, which can be achieved, for example, by using a small solar panel to recharge the battery. The question may arise whether LoRaMesher could be changed to a Time-Division Multiple Access (TDMA)-based protocol to improve energy efficiency. From a design point of view, adapting LoRaMesher to TDMA would imply very complex changes and the reuse of the current code base would be limited.


```

void sendLoRaMessage(void*) {
    int dataTablePosition = 0;

    for (;;) {
        if (radio.routingTableSize() == 0) {
            vTaskDelay(120000 / portTICK_PERIOD_MS);
            continue;
        }

        if (radio.routingTableSize() <= dataTablePosition)
            dataTablePosition = 0;

        LM_LinkedList<RouteNode>* routingTableList = radio.
            routingTableListCopy();

        uint16_t addr = (*routingTableList)[dataTablePosition]
            ->networkNode.address;

        dataTablePosition++;

        //Create packet and send it.
        radio.createPacketAndSend(addr, helloPacket, 1);

        //Increment data counter
        helloPacket->counter[0] = dataCounter++;

        //Wait 20 seconds to send the next packet
        vTaskDelay(120000 / portTICK_PERIOD_MS);
    }
}

```

Code 5. Code fragment from main.cpp that implements the send function.

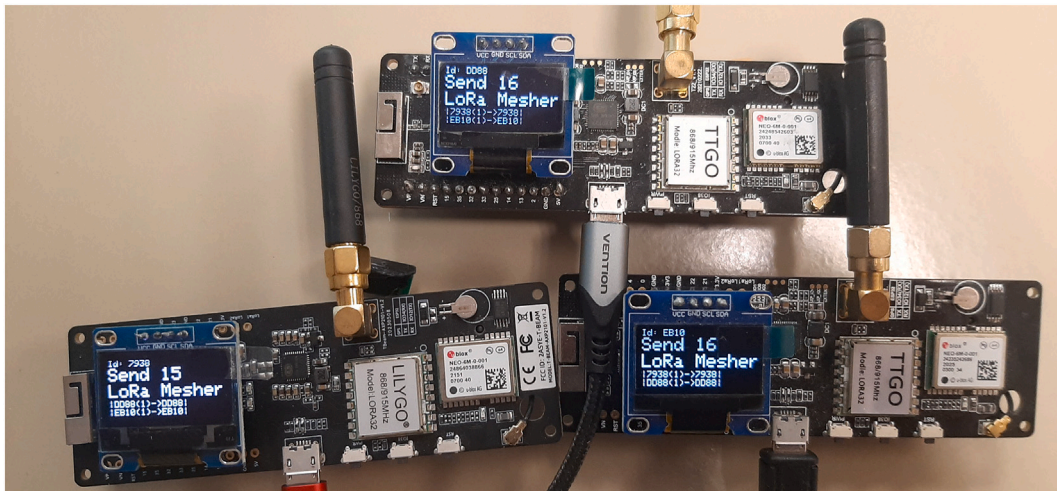


Fig. 2. T-Beam boards running the CounterAndDisplay example from the LoRaMesher library.

Regarding the use of LoRaMesher under different duty-cycle constraints, the operation is limited by the regulatory constraints rather than by the implementation of the library. In LoRaMesher, duty-cycle behavior is explicitly configurable via LM_DUTY_CYCLE in BuildOptions.h. When non-reliable transmission is used, packets are queued and transmitted when duty-cycle availability permits. For reliable transmission, retransmission behavior must be carefully tuned to the duty-cycle

regime. In particular, the MAX_TIMEOUTS parameter determines how re-transmissions are performed and should be selected empirically based on topology, traffic patterns, and regulatory limits.

With respect to traffic regimes in LoRa mesh networks, LoRa in general is targeted for low-data rates and the inherent constraints of LoRa regarding channel congestion and collisions under high traffic conditions should also be considered in a deployment of LoRaMesher

```
[322425][I][LoRaMesher.cpp:407] receivingRoutine(): [LoRaMesher] Receiving LoRa packet: Size: 20 bytes RSSI: -58 SNR: 9
[322437][V][LoRaMesher.cpp:662] processPackets(): [LoRaMesher] Size of Received Packets Queue: 1
[322445][I][LoRaMesher.cpp:790] printHeaderPacket(): [LoRaMesher] Packet received -- Size: 20 Src: 2F14 Dst: FFFF Id: 2
8 Type: 4 Via: 0 Seq_Id: 0 Num: 0
[322459][I][RoutingTableService.cpp:77] processRoute(): [LoRaMesher] Route packet from 2F14 with size 3
[322468][I][RoutingTableService.cpp:99] resetReceiveSNRRoutePacket(): [LoRaMesher] Reset Receive SNR from 2F14: 9
[322478][I][RoutingTableService.cpp:200] printRoutingTable(): [LoRaMesher] Current routing table:
[322487][I][RoutingTableService.cpp:214] printRoutingTable(): [LoRaMesher] 0 - E2B0 via E2B0 metric 1 Role 0
[322496][I][RoutingTableService.cpp:214] printRoutingTable(): [LoRaMesher] 1 - 2F14 via 2F14 metric 1 Role 0
[322506][I][RoutingTableService.cpp:214] printRoutingTable(): [LoRaMesher] 2 - C4F0 via C4F0 metric 1 Role 0
[322516][I][PacketQueueService.h:80] deleteQueuePacketAndPacket(): [LoRaMesher] Deleting packet
[322524][I][PacketQueueService.h:83] deleteQueuePacketAndPacket(): [LoRaMesher] Deleting packet queue
```

Fig. 3. Routing table at LoRaMesher node 0xF020 showing the addresses of the three other nodes in the network.

```
[161441][V][LoRaMesher.cpp:656] processPackets(): [LoRaMesher] Stack space unused after entering the task: 2036
[161451][V][LoRaMesher.cpp:657] processPackets(): [LoRaMesher] Free heap: 297120
ReceivedUserData_TaskHandle notify received
Queue receiveUserData size: 1
Packet arrived from E2B0 with size 840 bytes
Received data n° 0
---- Payload ---- Payload length in dataP: 6
Received data n° 00 -- 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24,
Received data n° 11 -- 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24,
Received data n° 22 -- 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24,
Received data n° 33 -- 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24,
Received data n° 44 -- 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24,
Received data n° 55 -- 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24,
---- Payload Done ----
[161837][I][LoRaMesher.cpp:790] printHeaderPacket(): [LoRaMesher] Packet send -- Size: 12 Src: F020 Dst: E2B0 Id
```

Fig. 4. A large payload of 840 bytes received at LoRaMesher node 0xF020.

```
[100865][I][LoRaMesher.cpp:790] printHeaderPacket(): [LoRaMesher] Packet received -- Size: 60 Src: E2B0 Dst: 2F14 Id: 16
Type: 19 Via: 2F14 Seq_Id: 0 Num: 10
[100879][I][LoRaMesher.cpp:890] processDataPacket(): [LoRaMesher] Data packet from E2B0, destination 2F14, via 2F14
[100889][V][LoRaMesher.cpp:895] processDataPacket(): [LoRaMesher] Data packet from E2B0 for me
[100897][V][LoRaMesher.cpp:960] processDataPacketForMe(): [LoRaMesher] Large payload Packet received
[100906][V][PacketFactory.h:45] createPacket(): [LoRaMesher] Creating packet with 12 bytes
[100914][I][PacketFactory.h:74] createPacket(): [LoRaMesher] Packet created with 12 bytes
[100922][I][LoRaMesher.h:710] setPackedForSend(): [LoRaMesher] Adding packet to Q_SP
[100930][I][LoRaMesher.h:712] setPackedForSend(): [LoRaMesher] Created packet to Q_SP
[100938][I][LoRaMesher.cpp:1066] addToSendOrderedAndNotify(): [LoRaMesher] Added packet to Q_SP, notifying sender task
```

Fig. 5. The fragment number 10 of a large payload is received at node 0x2F14.

nodes. For instance, in a permanent network deployment demonstrator⁵ that we operate with around ten LoRaMesher nodes, and where the reliable transfer is used, we observed that the amount of traffic sent has to be carefully determined to avoid saturating the network. Based on our experience, moderate traffic levels are advisable if the large messages and reliable transfer mode are used. While a temporarily high traffic load can be handled by the node's queues, a permanently high traffic scenario should be avoided since reliable transfer can increase network congestion from the nodes resending the lost packets. Concrete LoRaMesher PDR from deployments is reported in [16], ranging from 85% to 100% in the evaluated deployment scenarios, showing dependence on packet size and network topology, and end-to-end message delivery rates of the large and reliable message transfer in [19]. For practical deployments, experimentation will provide practitioners with valuable information for determining critical parameter values such as `LM_MAX_PACKET_SIZE` and `MAX_TIMEOUTS` of LoRaMesher's `BuildOptions.h` to achieve the desired behavior.

LoRaMesher does not include built-in security mechanisms such as authentication or encryption. When security is required, LoRaMesher can be extended by integrating existing cryptographic libraries. Depending on the security feature, the extension may be developed at

the application layer (for example, by encrypting the payload prior to invoking LoRaMesher's send functions) or library level. From a network traffic point of view, the effect of the additional cryptographic data to be communicated in the packets and the need for additional control messages should be taken into account.

Regarding the scalability of LoRaMesher with regard to the number of nodes, from a design perspective, LoRaMesher does not impose a strict implementation-level limit on the number of nodes, beyond the size of the node identifier field. Routing information is stored in dynamically allocated data structures, and the routing task itself is not directly affected by the number of nodes. Therefore, in practice, the scalability is not determined by memory constraints but by network-level factors such as traffic volume, routing packet overhead, and topology. As the number of nodes increases, routing and data traffic can quickly saturate the network. In practice, this limits the effective network size well before any implementation-level constraints are reached.

LoRaMesher is available to the community via an open Git repository, which, at the time of writing, has more than 240 stars. There is an active Discord community with more than 90 participants discussing practical use cases, including livestock monitoring, water-quality monitoring, underground installations requiring multi-hop communication, and temporary ad-hoc networks for events. The use of LoRaMesher in scientific publications and the ongoing community adoption highlights the library's relevance for both research and operational deployments.

⁵ <https://tomir.ac.upc.edu/loraucp/index.php>.

```

Send data packet n° 0 to E2B0 (0)
[ 26390][V][LoraMesher.cpp:810] sendReliablePacket(): [LoRaMesher] Sending reliable payload with 840 bytes to E2B0
[ 26403][V][PacketFactory.h:45] createPacket(): [LoRaMesher] Creating packet with 12 bytes
[ 26411][I][PacketFactory.h:74] createPacket(): [LoRaMesher] Packet created with 12 bytes
[ 26419][V][LoraMesher.cpp:848] sendReliablePacket(): [LoRaMesher] Payload Size: 88
[ 26426][V][PacketFactory.h:45] createPacket(): [LoRaMesher] Creating packet with 100 bytes
[ 26435][I][PacketFactory.h:74] createPacket(): [LoRaMesher] Packet created with 100 bytes
[ 26443][V][LoraMesher.cpp:848] sendReliablePacket(): [LoRaMesher] Payload Size: 88
[ 26450][V][PacketFactory.h:45] createPacket(): [LoRaMesher] Creating packet with 100 bytes
[ 26458][I][PacketFactory.h:74] createPacket(): [LoRaMesher] Packet created with 100 bytes
[ 26466][V][LoraMesher.cpp:848] sendReliablePacket(): [LoRaMesher] Payload Size: 88
[ 26474][V][PacketFactory.h:45] createPacket(): [LoRaMesher] Creating packet with 100 bytes
[ 26482][I][PacketFactory.h:74] createPacket(): [LoRaMesher] Packet created with 100 bytes
[ 26490][V][LoraMesher.cpp:848] sendReliablePacket(): [LoRaMesher] Payload Size: 88
[ 26498][V][PacketFactory.h:45] createPacket(): [LoRaMesher] Creating packet with 100 bytes
[ 26506][I][PacketFactory.h:74] createPacket(): [LoRaMesher] Packet created with 100 bytes
[ 26514][V][LoraMesher.cpp:848] sendReliablePacket(): [LoRaMesher] Payload Size: 88
[ 26521][V][PacketFactory.h:45] createPacket(): [LoRaMesher] Creating packet with 100 bytes
[ 26529][I][PacketFactory.h:74] createPacket(): [LoRaMesher] Packet created with 100 bytes
[ 26537][V][LoraMesher.cpp:848] sendReliablePacket(): [LoRaMesher] Payload Size: 88
[ 26545][V][PacketFactory.h:45] createPacket(): [LoRaMesher] Creating packet with 100 bytes
[ 26553][I][PacketFactory.h:74] createPacket(): [LoRaMesher] Packet created with 100 bytes
[ 26561][V][LoraMesher.cpp:848] sendReliablePacket(): [LoRaMesher] Payload Size: 88
[ 26569][V][PacketFactory.h:45] createPacket(): [LoRaMesher] Creating packet with 100 bytes
[ 26577][I][PacketFactory.h:74] createPacket(): [LoRaMesher] Packet created with 100 bytes
[ 26585][V][LoraMesher.cpp:848] sendReliablePacket(): [LoRaMesher] Payload Size: 88
[ 26592][V][PacketFactory.h:45] createPacket(): [LoRaMesher] Creating packet with 100 bytes
[ 26601][I][PacketFactory.h:74] createPacket(): [LoRaMesher] Packet created with 100 bytes
[ 26609][V][LoraMesher.cpp:848] sendReliablePacket(): [LoRaMesher] Payload Size: 88
[ 26616][V][PacketFactory.h:45] createPacket(): [LoRaMesher] Creating packet with 100 bytes
[ 26624][I][PacketFactory.h:74] createPacket(): [LoRaMesher] Packet created with 100 bytes
[ 26632][V][LoraMesher.cpp:848] sendReliablePacket(): [LoRaMesher] Payload Size: 48
[ 26640][V][PacketFactory.h:45] createPacket(): [LoRaMesher] Creating packet with 60 bytes
[ 26648][I][PacketFactory.h:74] createPacket(): [LoRaMesher] Packet created with 60 bytes
[ 26656][V][LoraMesher.cpp:1573] calculateTimeout(): [LoRaMesher] No RTT data yet, using default timeout: 25000 ms (hops=1)
[ 26668][V][LoraMesher.cpp:1607] addTimeout(): [LoRaMesher] Timeout set to 25 s for addr E2B0
[ 26677][I][LoraMesher.h:710] setPackedForSend(): [LoRaMesher] Adding packet to Q_SP
[ 26684][I][LoraMesher.h:712] setPackedForSend(): [LoRaMesher] Created packet to Q_SP
[ 26692][I][LoraMesher.cpp:1066] addToSendOrderedAndNotify(): [LoRaMesher] Added packet to Q_SP, notifying sender task

```

Fig. 6. Node 0xC4F0 prepares a large data transfer.

5. Conclusions

LoRaMesher is an open-source, MIT-licensed library that enables multi-hop LoRa mesh networking on ESP32-based IoT devices. It implements a proactive distance-vector routing protocol, supports data fragmentation and reliable data transfer, and integrates with the PlatformIO development environment. As a community-driven project, we encourage further experimental evaluations in untested scenarios and new applications to continue advancing the capabilities of LoRa mesh technology.

CRedit authorship contribution statement

Joan Miquel Solé: Software, Investigation Data curation. **Felix Freitag:** Supervision, Methodology, Investigation, Funding acquisition, Data curation, Conceptualization. **Roger Baig:** Writing – review & editing, Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the **Ministry of Science and Innovation of the Spanish Government through the State Research Agency (AEI)** under projects **PID2023-146066OB-I00** AEI 2023 Knowledge Generation Projects and **PDC2023-145809-I0** AEI 2023 Knowledge Generation Projects, financed by MICIU/AEI/10.13039/501100011033/ and by FEDER, EU.

References

- [1] Pueyo Centelles R, Freitag F, Meseguer R, Navarro L. Beyond the star of stars: an introduction to multihop and mesh for LoRa and LoRaWAN. *IEEE Pervasive Comput* 2021;20(2):63–72. <https://doi.org/10.1109/MPRV.2021.3063443>
- [2] Wong AW-L, Goh SL, Hasan MK, Fattah S. Multi-hop and mesh for lora networks: recent advancements, issues, and recommended applications. *ACM Comput Surv Jan* 2024;56(6). <https://doi.org/10.1145/3638241>
- [3] Li C, Cao Z. LoRa networking techniques for large-scale and long-term IoT: a down-to-top survey. *ACM Comput Surv* 2023;55(3):1–36. <https://doi.org/10.1145/3494673>
- [4] Álamos J, Kietzmann P, Schmidt TC, Wählich M. DSME-LoRa: seamless long-range communication between arbitrary nodes in the constrained IoT. *ACM Trans Sen Netw* Nov 2022;18(4). <https://doi.org/10.1145/3552432>
- [5] Leonardi L, Lo Bello L, Patti G. MRT-LoRa: a multi-hop real-time communication protocol for industrial IoT applications over LoRa networks. *Comput Commun* 2023;199:72–86. <https://doi.org/10.1016/j.comcom.2022.12.013>. <https://www.sciencedirect.com/science/article/pii/S0140366422004637>.
- [6] Muthanna MSA, Muthanna A, Rafiq A, Hammoudeh M, Alkanhel R, Lynch S, Abd El-Latif AA. Deep reinforcement learning based transmission policy enforcement and multi-hop routing in QoS aware LoRa IoT networks. *Comput Commun* 2022;183:33–50. <https://doi.org/10.1016/j.comcom.2021.11.010>. <https://www.sciencedirect.com/science/article/pii/S0140366421004394>.
- [7] Berto R, Napoletano P, Savi M. A lora-based mesh network for peer-to-peer long-range communication. *Sensors* 2021;21(13). <https://doi.org/10.3390/s21134314>. <https://www.mdpi.com/1424-8220/21/13/4314>.
- [8] Arratia B, Rosas E, Calafate CT, Cano J-C, Cecilia JM, Manzoni P. AllLoRa: empowering environmental intelligence through an advanced LoRa-based IoT solution. *Comput Commun* 2024;218:44–58. <https://doi.org/10.1016/j.comcom.2024.02.014>. <https://www.sciencedirect.com/science/article/pii/S0140366424000641>.
- [9] Abadade Y, Temouden A, Bamoumen H, Benamar N, Chtouki Y, Hafid AS. A comprehensive survey on TinyML. *IEEE Access* 2023;11:96892–922. <https://doi.org/10.1109/ACCESS.2023.3294111>
- [10] Sudharsan B, Breslin JG, Tahir M, Intizar Ali M, Rana O, Dustdar S, Ranjan R. OTA-TinyML: over the AIR deployment of TinyML models and execution on IoT devices. *IEEE Internet Comput* 2022;26(3):69–78. <https://doi.org/10.1109/MIC.2021.3133552>
- [11] Qi P, Chiaro D, Piccialli F. Small models, big impact: a review on the power of lightweight federated learning. *Future Gener Comput Syst* 2025;162:107484. <https://doi.org/10.1016/j.future.2024.107484>. <https://www.sciencedirect.com/science/article/pii/S0167739X24004400>.

- [12] Meshtastic: open source hiking, pilot, skiing and secure GPS mesh communicator; 2020. <https://meshtastic.org/> [accessed: December 2025].
- [13] Neocortec home page. <https://neocortec.com/> [accessed: January 2026].
- [14] Orama-system home page. <https://orama-system.com/> [accessed: January 2026].
- [15] Pueyo Centelles R. Towards LoRa mesh networks for the IoT, [Ph.D. thesis], Universitat Politècnica de Catalunya, Computer Architecture Department, Barcelona, Spain. Nov 2021.
- [16] Solé JM, Centelles RP, Freitag F, Meseguer R. Implementation of a LoRa mesh library. *IEEE Access* 2022;10:113158–71. <https://doi.org/10.1109/ACCESS.2022.3217215>
- [17] Centelles RP, Meseguer R, Freitag F, Viñas RB, Navarro L. A minimalistic distance-vector routing protocol for LoRa mesh networks. *IEEE Access* 2024;1. <https://doi.org/10.1109/ACCESS.2024.3443605>
- [18] Miquel Solé J, Pueyo Centelles R, Freitag F, Meseguer R, Baig R. Middleware for distributed applications in a lora mesh network. *ACM Trans Embed Comput Syst* Jul 2025;24(4). <https://doi.org/10.1145/3747295>
- [19] Miquel Solé J, Pueyo Centelles R, Freitag F, Meseguer R, Baig Viñas R. Large and reliable data transfer service for LoRa mesh network applications. *Comput Commun* 2026;248:108404. <https://doi.org/10.1016/j.comcom.2025.108404>. <https://www.sciencedirect.com/science/article/pii/S0140366425003615>.